



Life lessons and datacenter performance analysis

Dan Ardelean
Amer Diwan
Rick Hank
Christian Kurmann
Balaji Raghavan
Matt Seegmiller

The need to solve performance crimes

Performance crimes are anything that unnecessarily increase

- Latency
- Resource usage

Performance crimes...

- degrade end user experience
- waste valuable resources (energy, cost, etc.)

Solving performance crimes is a necessity not a luxury

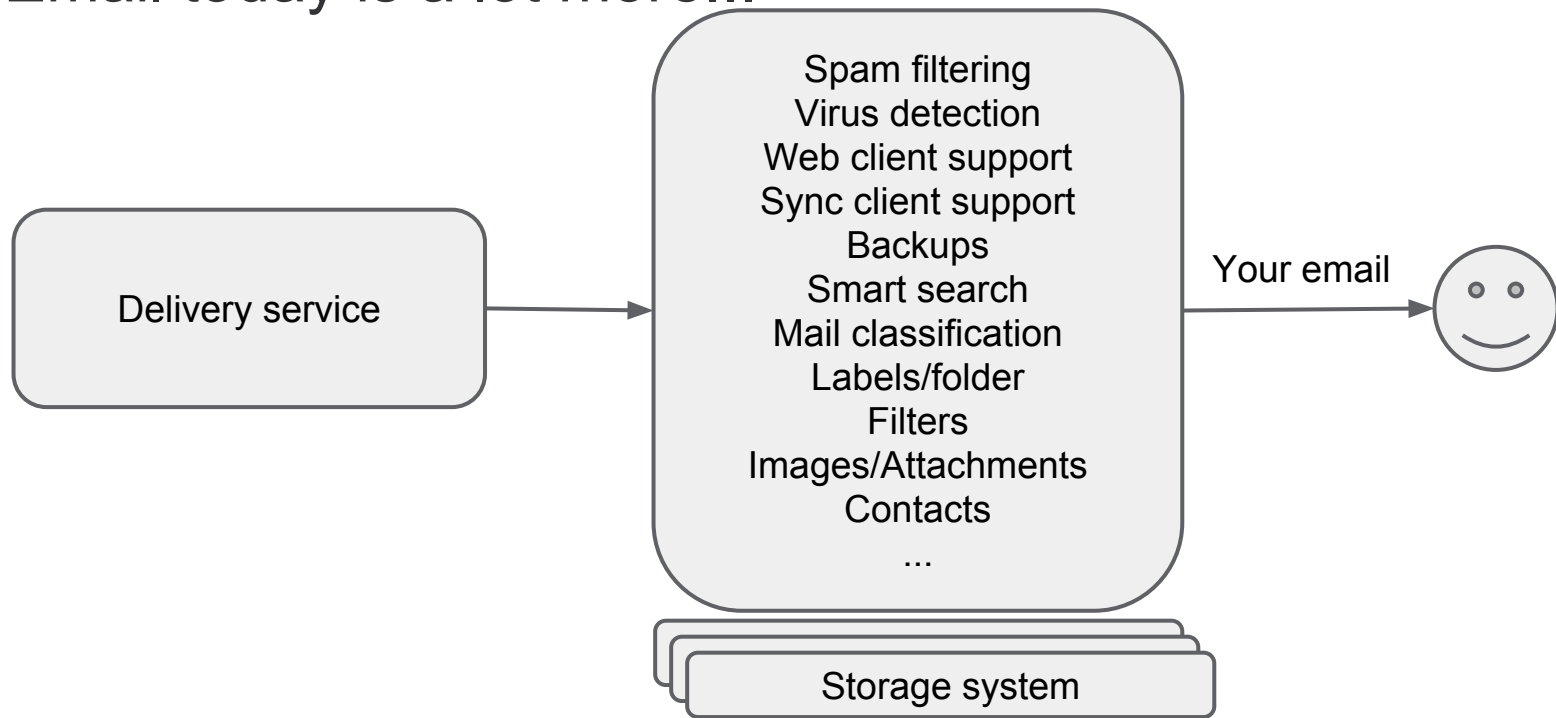
This talk shares our experiences in solving performance crimes in Gmail

But email is such a simple app ... right?



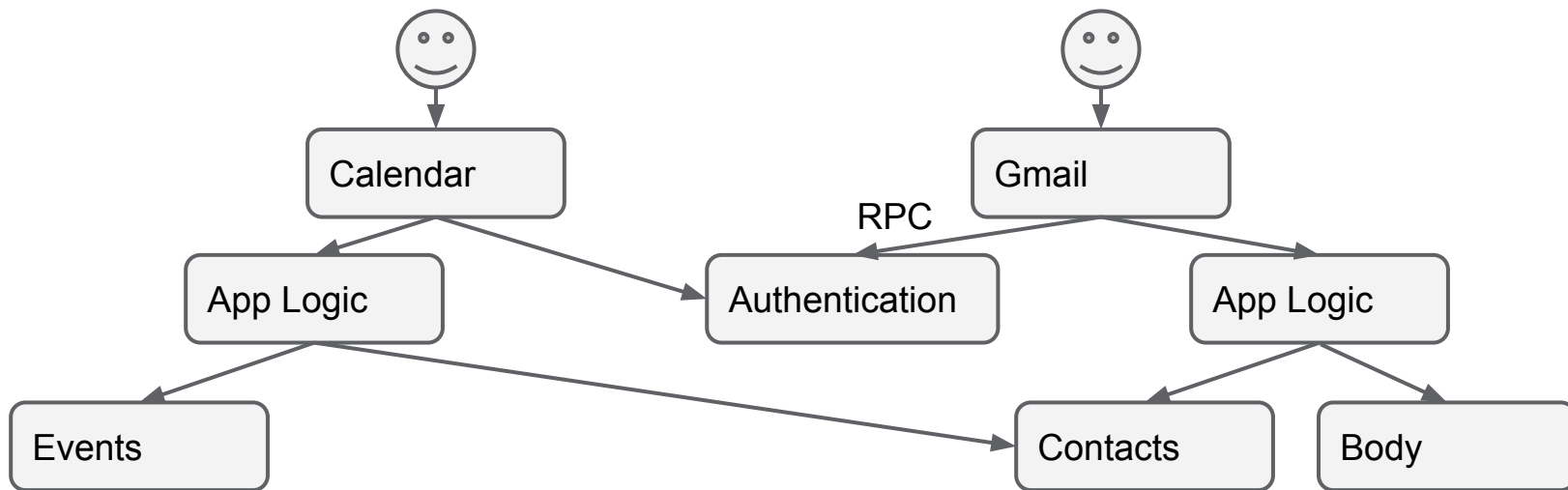
We have all used this...in the 1990s!

Email today is a lot more...



To enable sharing across applications (e.g., contacts) each component is a service

Each component is a service running in its own processes



Provides modularity, parallelism, reliability



Lesson 1:

No RPC left behind

Each user request involves $O(100)$ RPCs

We cannot ignore the rarely slow RPCs

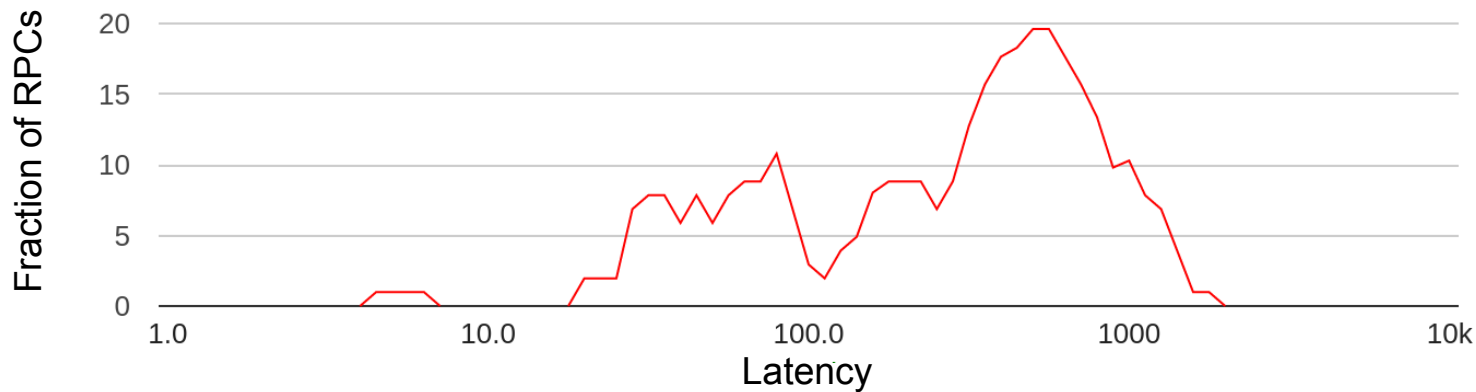
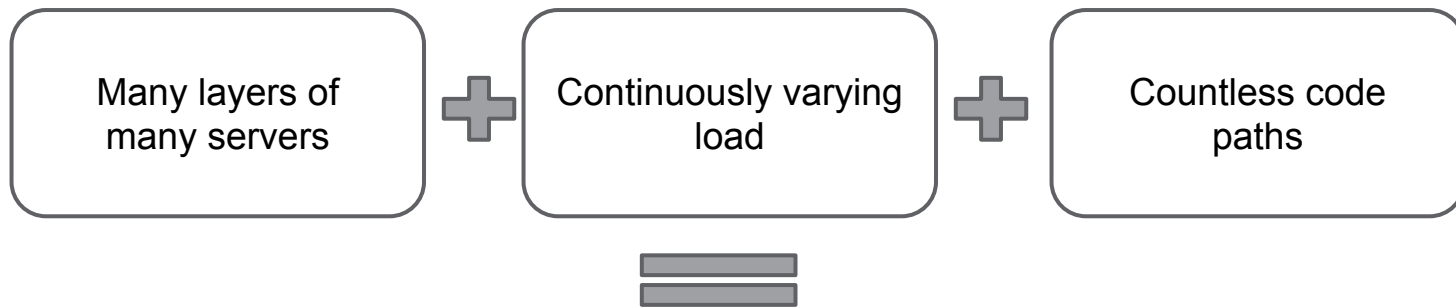
- 1/100 slow RPC affects 63% of the requests
- 1/1M slow RPC affects 0.01% of the requests

We cannot ignore the rarely slow requests

- 1/1M event affects $O(10M)$ requests daily

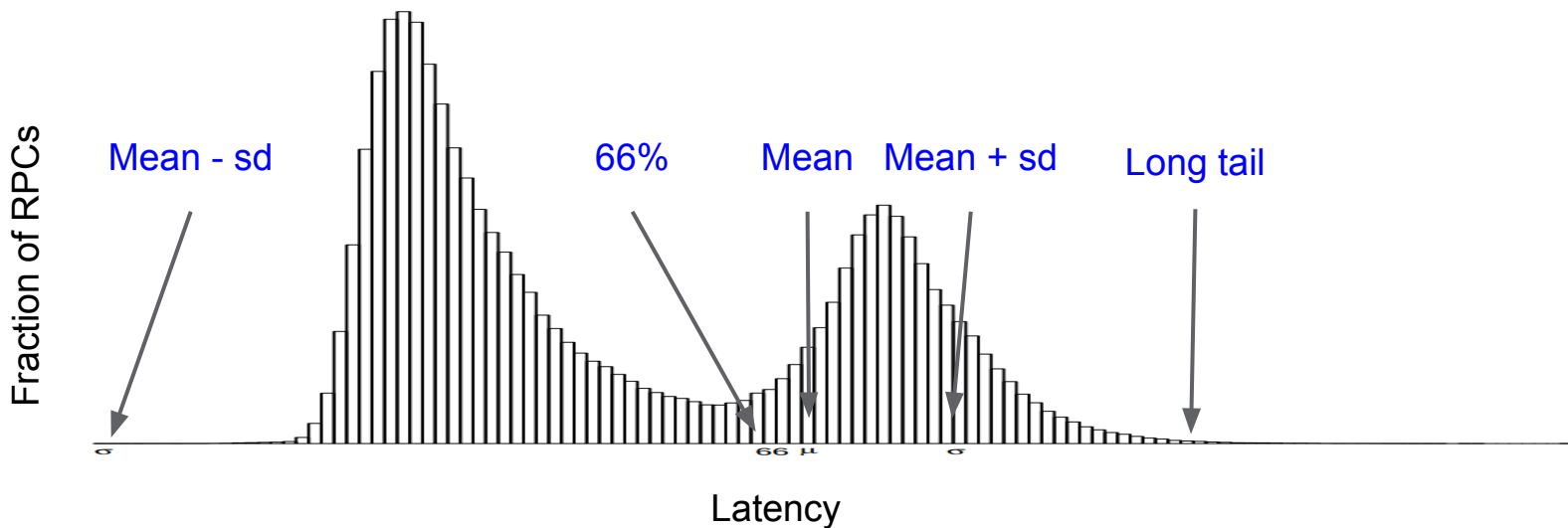
Blue moon...daily!

Latency of RPCs follows a complex distribution



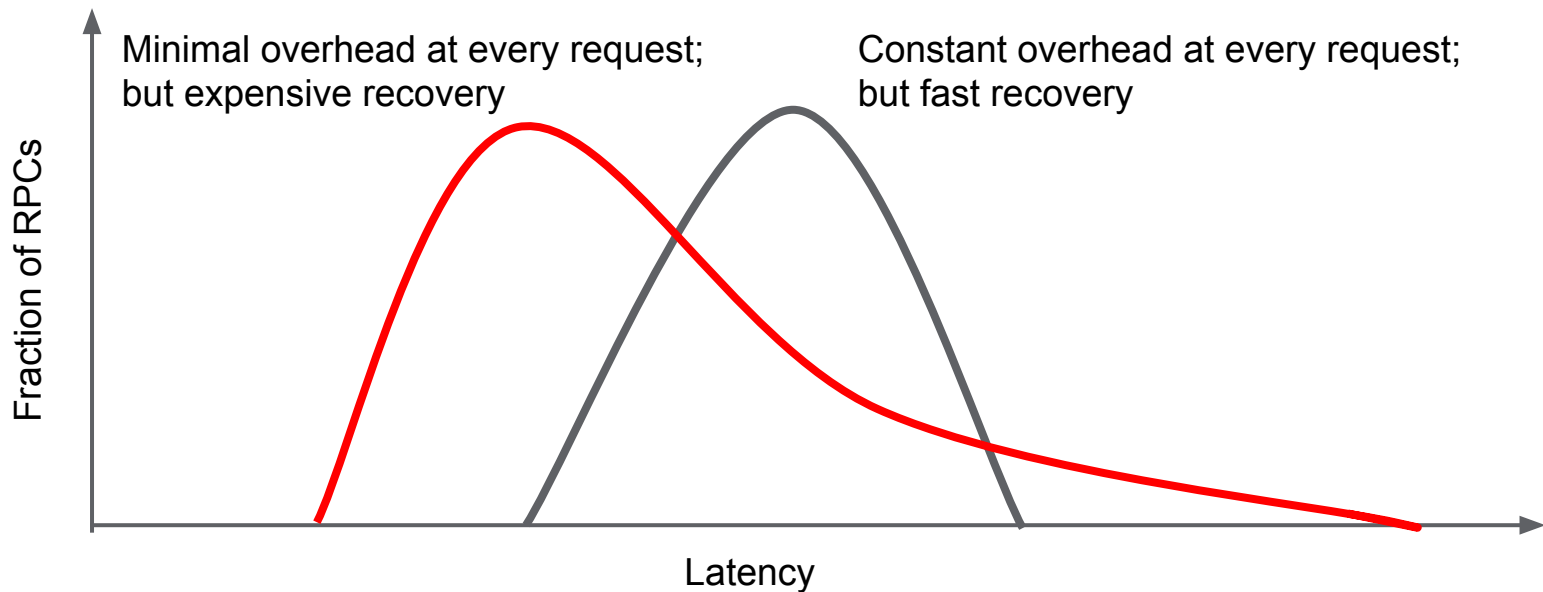
Even for “simplest” components data is rarely normal

Log scale latency distribution of a critical (but “simple”) component at Google



Abnormal is the new normal!

When is better actually better?



A tighter distribution may be better than a better median

Challenges

Normal distribution are rare at Google

- Optimizing only for the “common case” is inadequate

Many statistical *truths* assume properties of the data

- Ensure that analysis is **appropriate** for the shape of the data

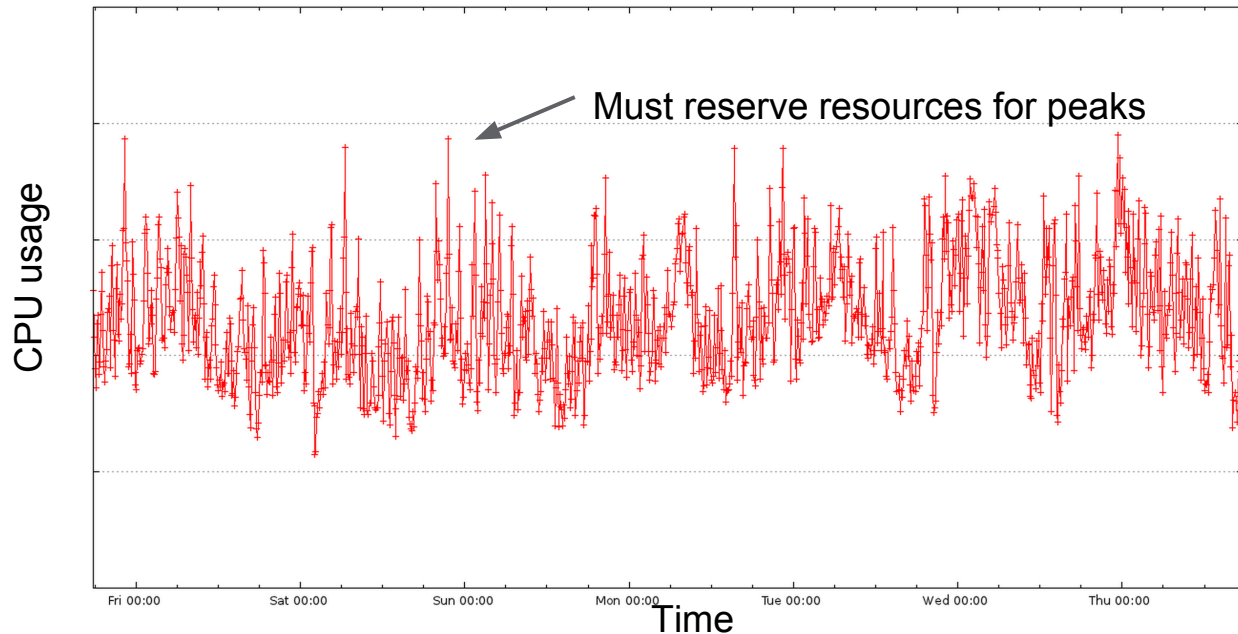
But...don't invent statistics; just use it correctly!



Lesson 2:

Prepare for the storm

Long tail also shows up in resource usage

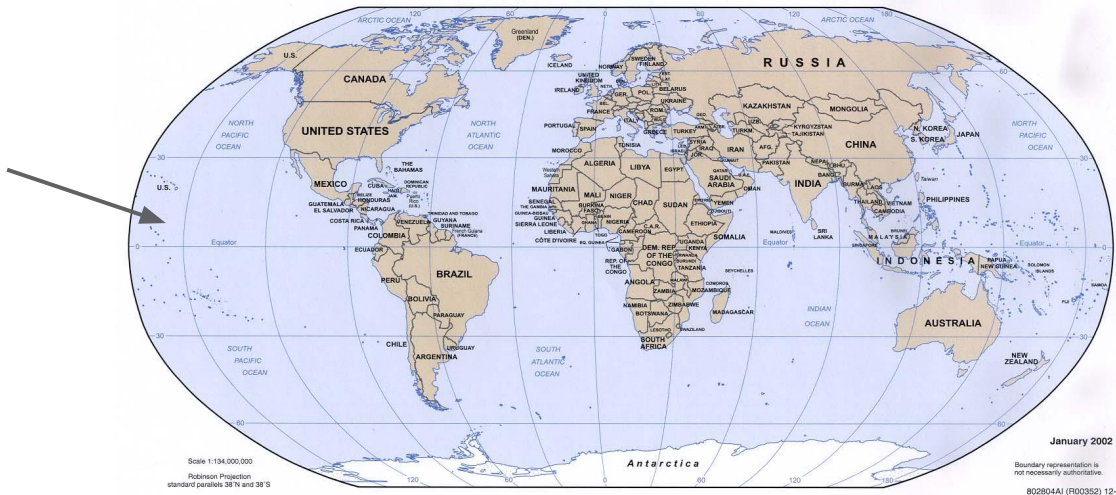


Two causes for peaks: (i) load; (ii) unusual events

Cause 1: load varies over time

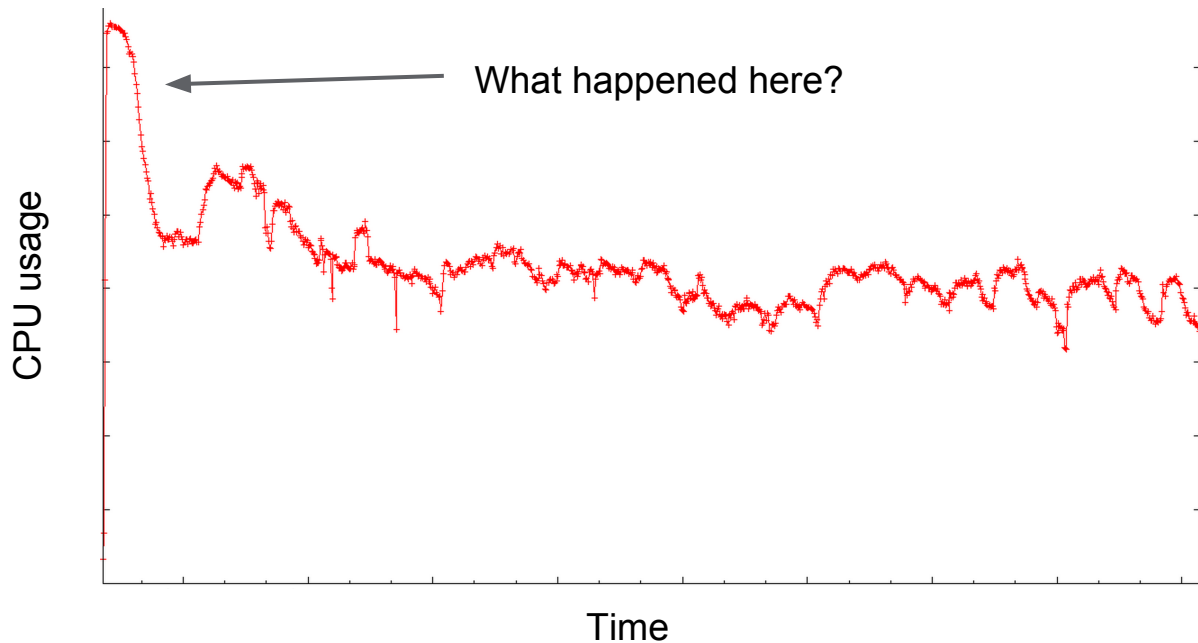
Users are not “randomly” spread out over time zones

Who is using Gmail here?



Working overlaps between zones (e.g., NA and European)
cause large usage peaks

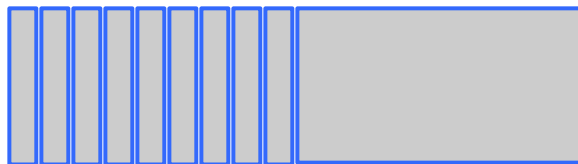
Cause 2: storms!



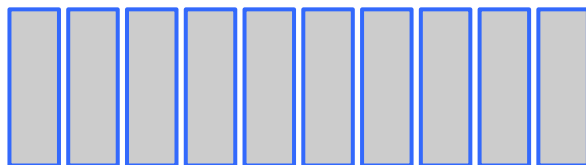
Must plan for hardware and software updates

Consequences of Lessons 1 and 2

Aggregate metrics (e.g., mean latency) cannot distinguish between:



and



We must reason with traces for long-tail events

Challenges with traces

- Different traces may use different clocks
 - We use large amounts of data to do time alignment after the fact
- Reasoning is hard and laborious
 - Traces from a single machine may be 100K events per second
 - We use a language based on temporal logic to reason over traces
- Need to coordinate tracing across machines
 - Use coordinated-bursty tracing

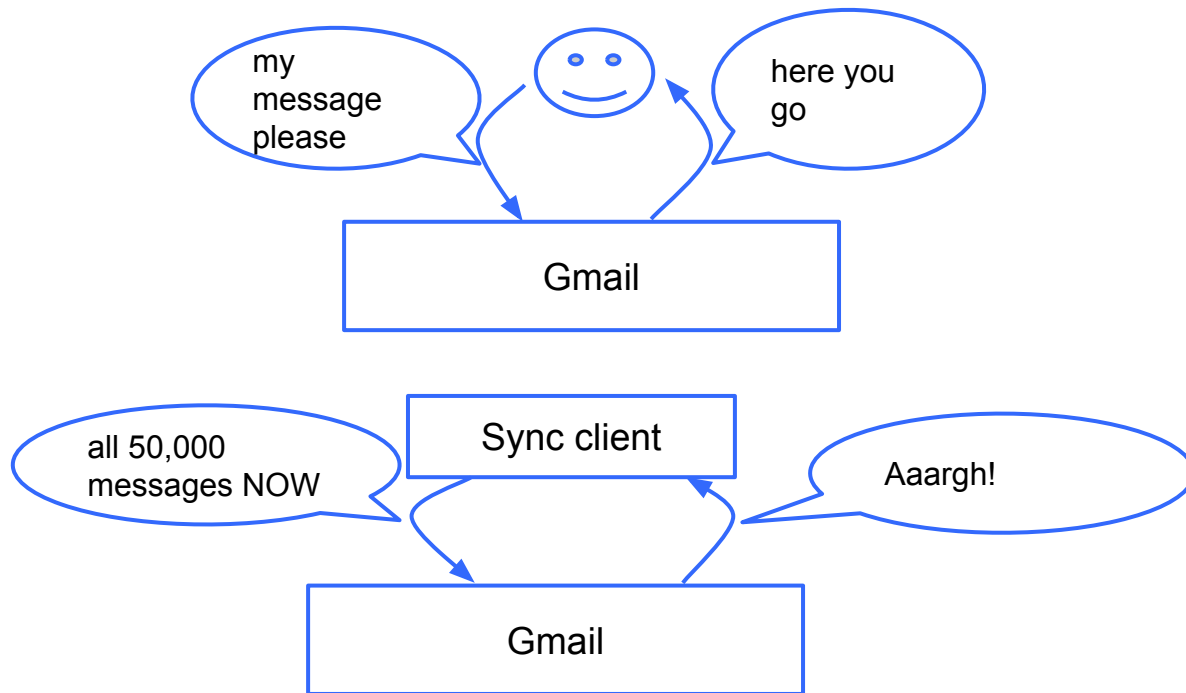
Profiling tools are (mostly) ok; now we must invest in tracing



Lesson 3:

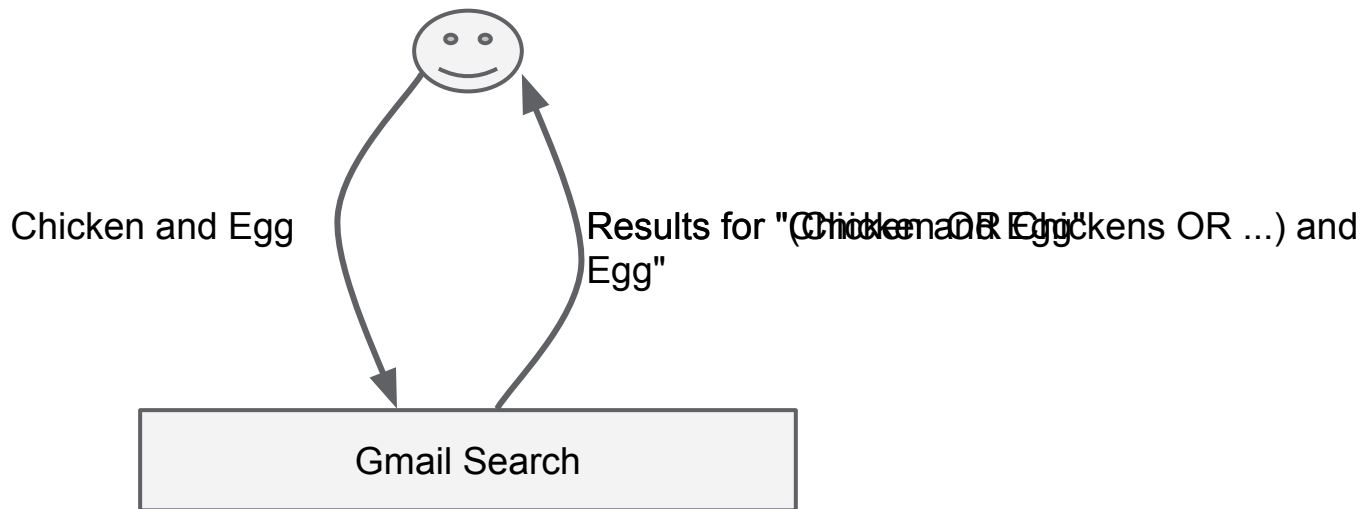
Ask nicely!

Is your pattern of access reasonable?



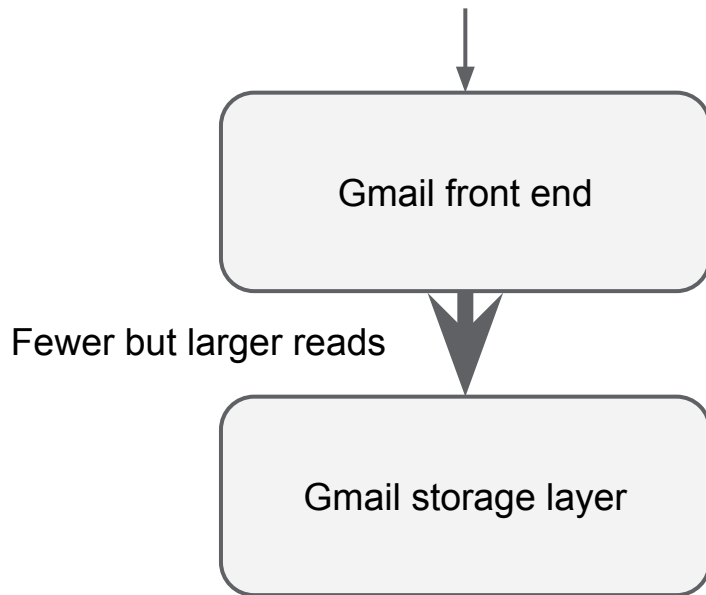
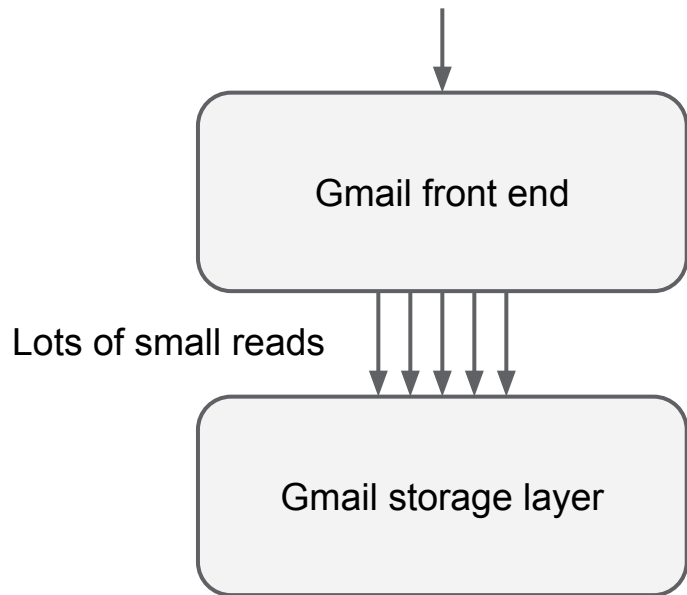
A single not-so-nice request can degrade many requests

Some problems are more subtle



Query rewriting can dramatically inflate a small request

Any layer can potentially overwhelm the next layer



Requests to a layer should match its strengths

Challenges

Our APIs capture functionality, not performance

How do we know that a method that we are calling...

- ...makes expensive RPC calls?
- ...acquires locks?
- ...performs IO?

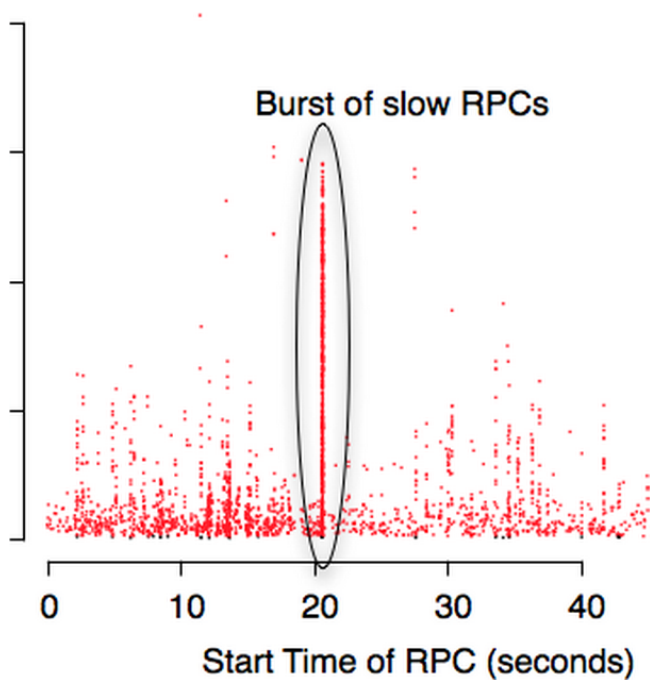
Can we express and reason over performance in our APIs?



Lesson 4:

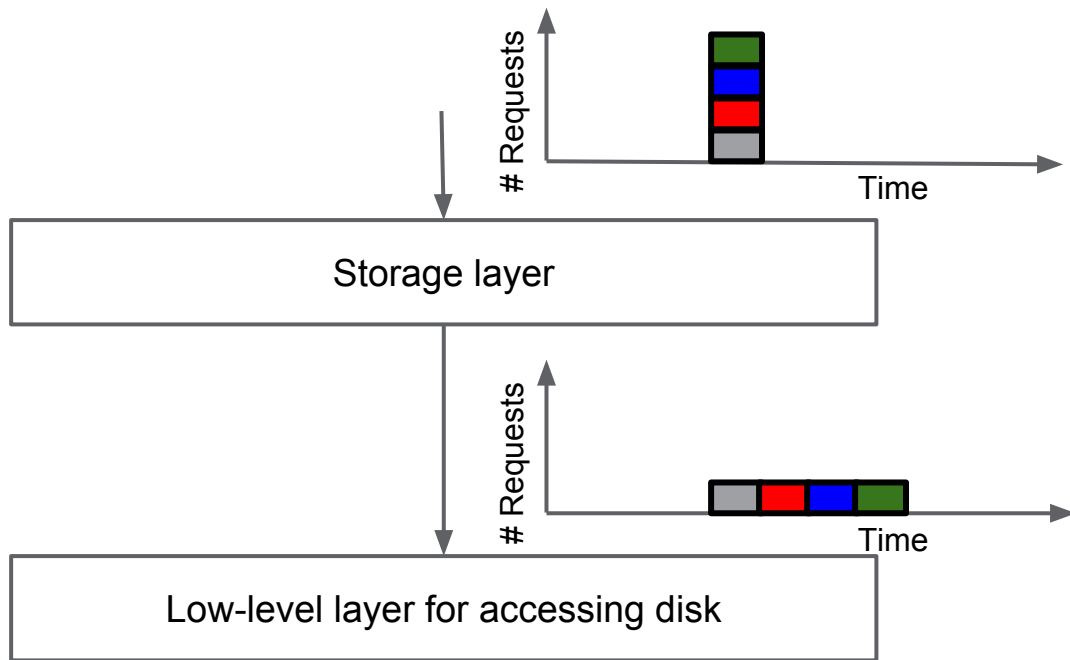
Share!

Lock contention often causes long-tail latency



A burst can cause contention when there is normally none

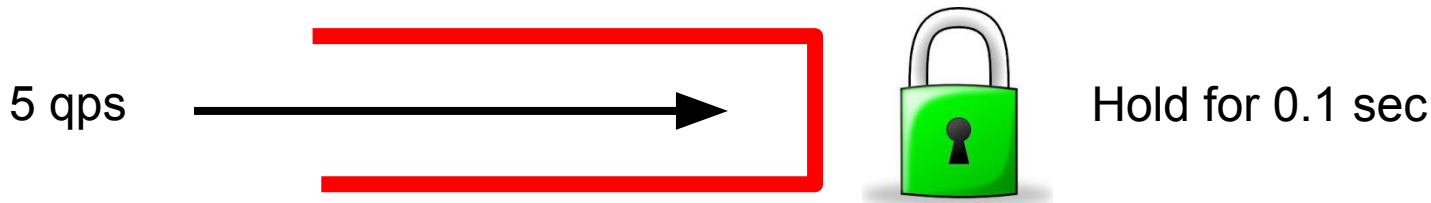
Layer interactions may aggravate or alleviate contention



An inefficient layer presents an easier request stream to next layer

Remember Little's law!

If processing a request holds a lock for 100ms
we cannot process more than 10 requests per second



Average queue depth: $5 * 0.1 = \frac{1}{2}$

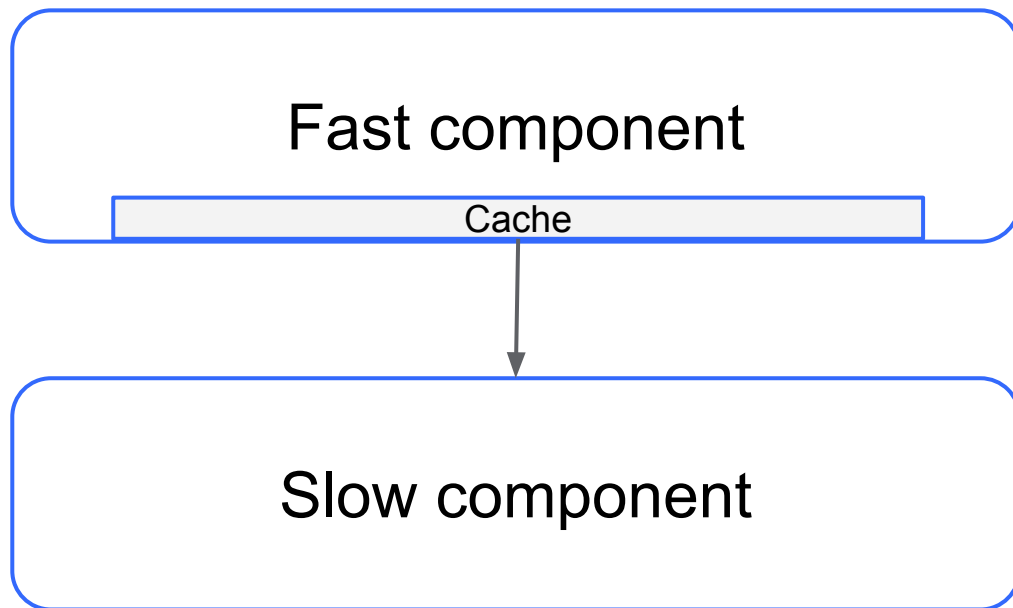
If we want to double parallelism we must halve holding time



Lesson 5:

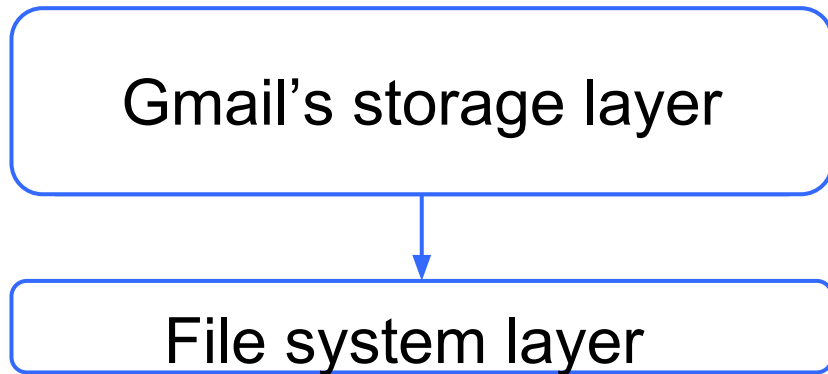
Confront weaknesses

Caches can hide the latency of “weak” components ...



...but it is often much better to fix the root cause!

Why should we care if caches reduce latency?



Glass is half full...

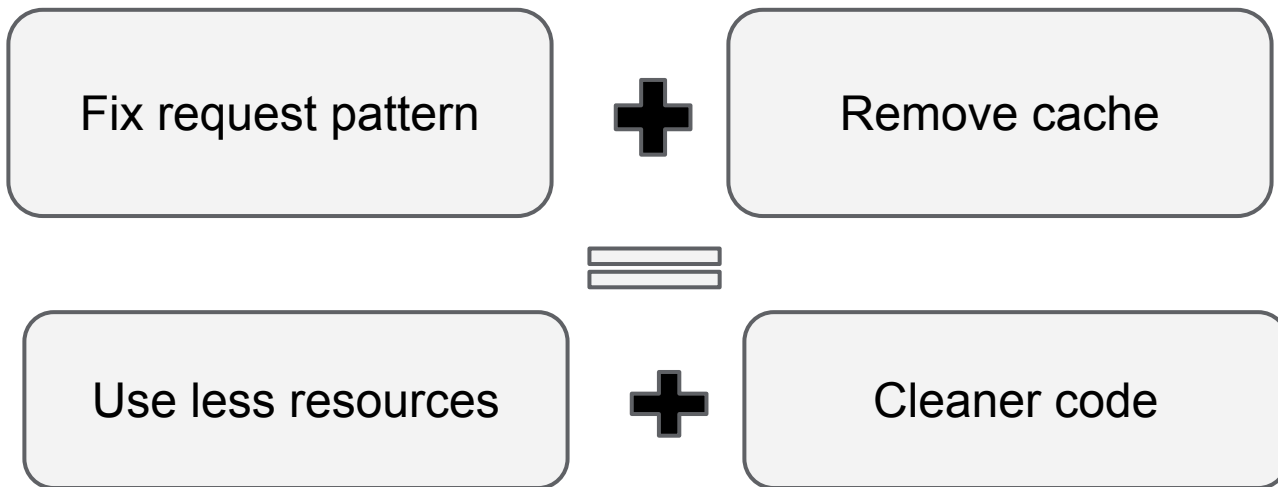
Wow, a 98.5% cache hit rate!

Glass is half empty...

Wow, why are we asking for the same data 98.5% of the time?

Fix the problem not the symptom

If cache is performing too well, something is wrong with the requests



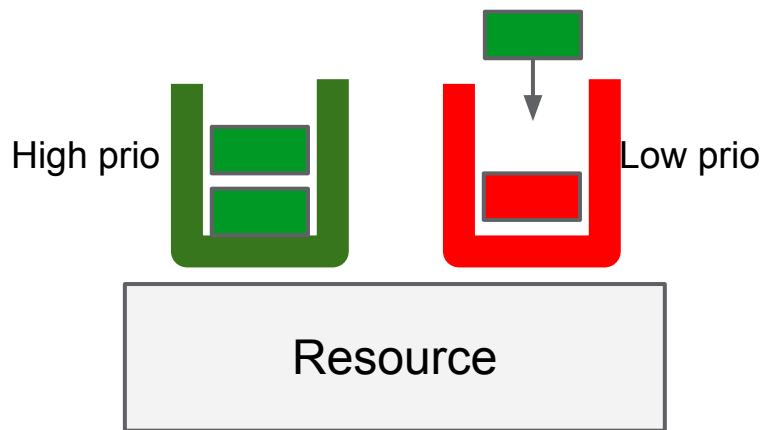
Caches are great abuse detectors!



Lesson 6:

Get your priorities right

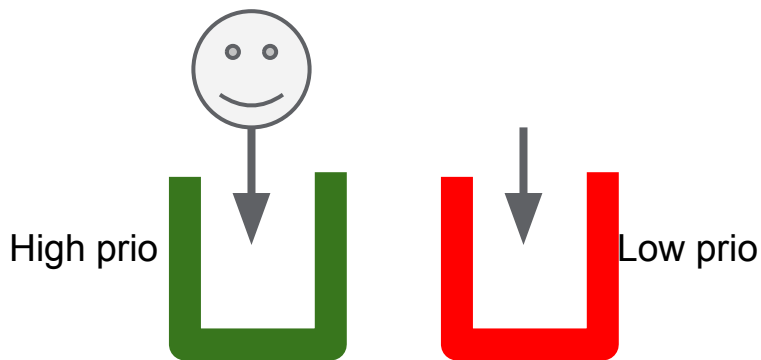
Priorities only matter in a crunch



Long queues: big problem!

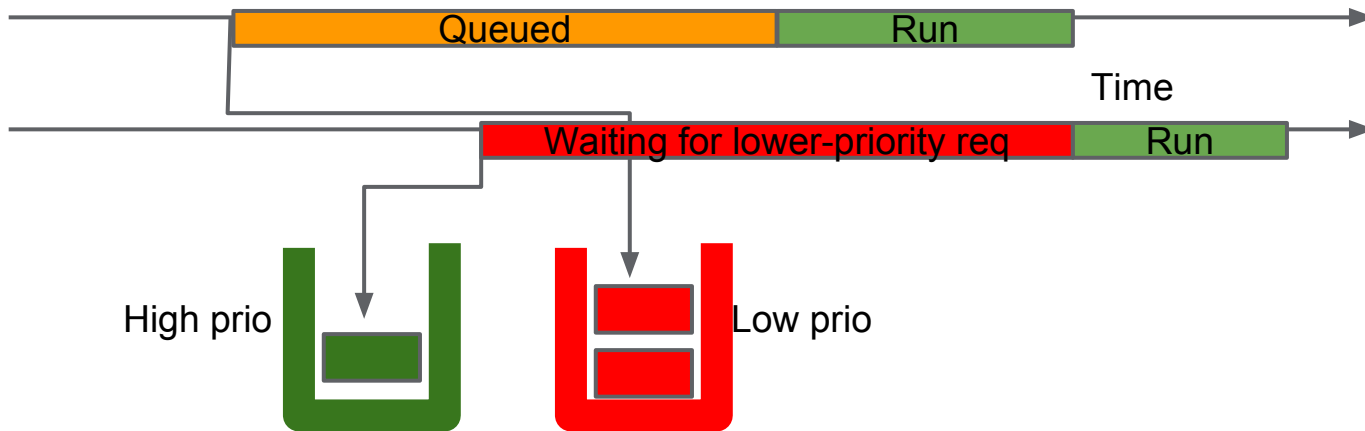
Overprovisioned resources mask poor priority settings

Setting priorities is hard!



First try: put only user-facing requests in high priority

...but that can easily backfire when there are dependencies



Avoiding priority inversion makes for a complex priority model



Lesson 7:

Question everything

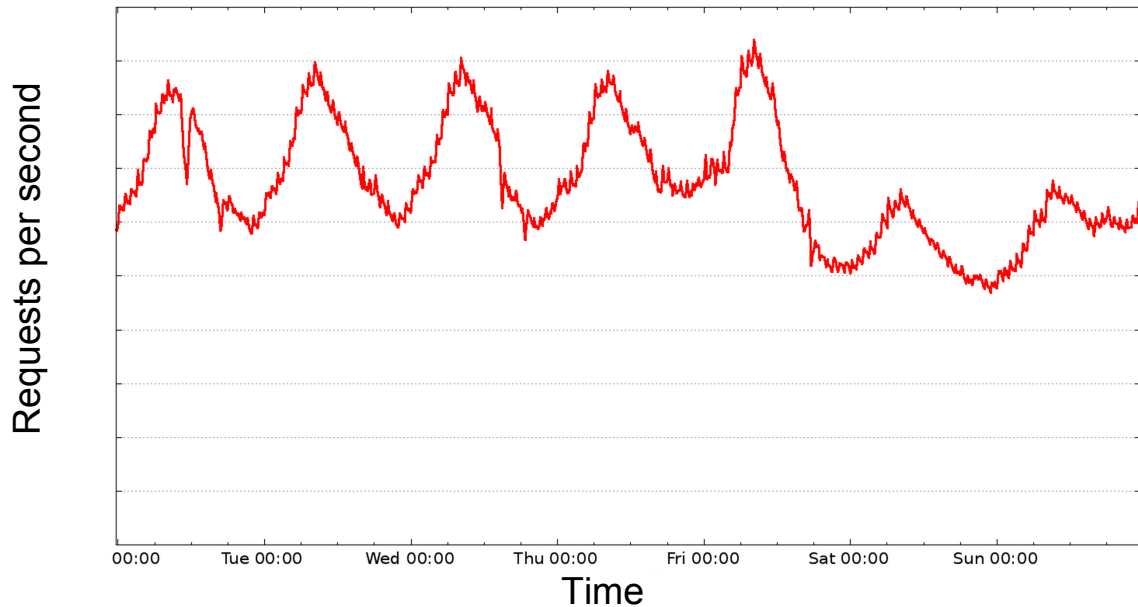
Suspect every layer, every component, every lock, ...

But even more so, look every gift horse in the mouth



If data looks too good to be true, it is probably wrong!

Rookie mistake: Compare Monday data to Sunday data!



The load varies day to day, hour to hour!

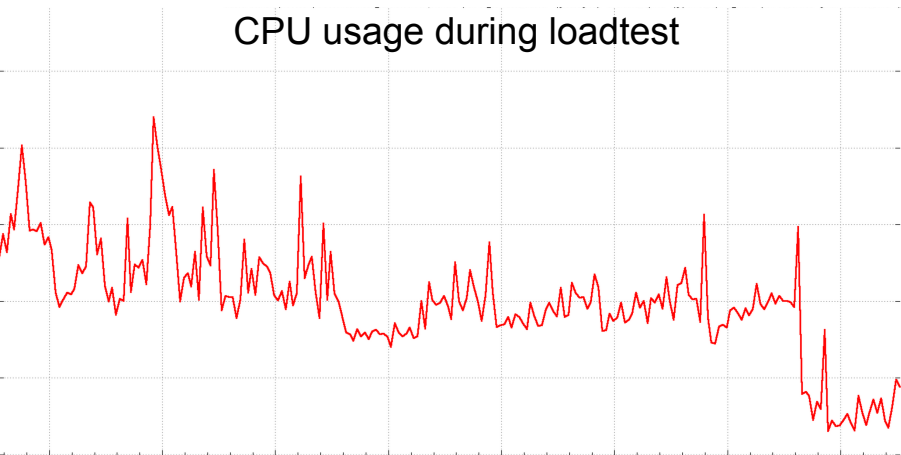


Lesson 8

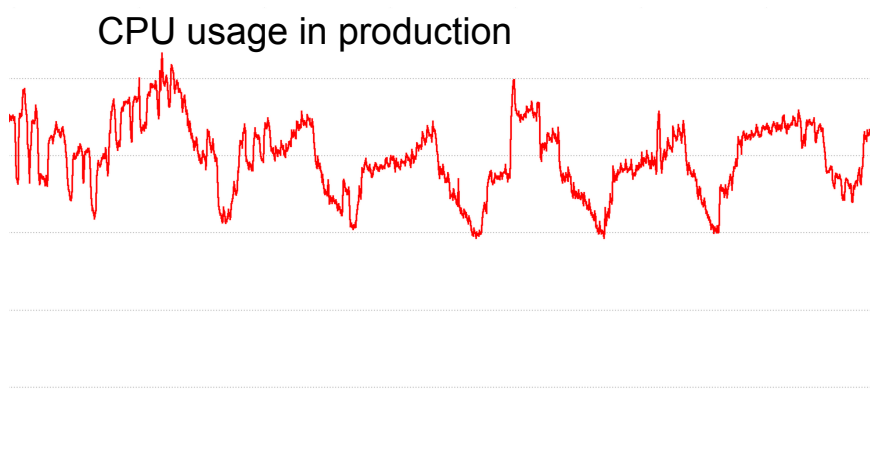
Real life is real life; tests are tests!

Loadtest versus production

CPU usage during loadtest



CPU usage in production



The loadtest executes same binaries...but the load is different

Why the differences?

- Real user pattern is more complex than what we can synthesize
 - Users use a variety of email clients which affect requests
 - Users are different in their mailbox size and usage
 - e.g., Googlers are 10x more of everything compared to “average” user
 - Users have different settings (e.g., filters) and usage styles (clean-inbox versus everything-in-inbox)
 - ...
- The loadtest attempts to model these...
 - but it cannot possibly model everything

Many performance problems must be debugged in production

Debugging in production



If only I knew X I could get to the bottom of this puzzle

Sure... but only if

- Recording X does not violate any compliance, contractual, and privacy promises to our users
- It is carefully reviewed so that we do not introduce bugs or regressions
- And it still needs to go through a careful rollout process

Challenge is to infer what we really need from what we have

Call to action

- Trace analysis tools
 - How to **combine** and analyze diverse sources of data
 - How can we **derive** high-level knowledge from low-level data?
- Performance APIs
 - How can we **express** and **check** performance specifications?
- Focus on analyzing large production systems when possible
 - How can our students **learn** from and **impact** real systems?

Are you up to it?